Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота № 3

з дисципліни «Основи розробки розподіленого програмного забезпечення мовою C++»

Тема: «Багатозадачність та багатопоточність»

Виконав:
студенти групи ІС-41
Авдєєв Єгор
Гаврилюк Артем

Перевірив:
Фещенко Кирило Юрійович

Київ 2026

Мета роботи

Ознайомитися з концепціями багатозадачності та багатопоточності сучасних операційних систем. Набути практичних навичок створення програм із паралельно виконуваними функціями, керування потоками та організації їх взаємодії з графічним інтерфейсом Windows.

Завдання роботи

Реалізувати багатопотокову програму на платформі Win32. Розглянути створення та завершення потоків, передавання їм параметрів, сповіщення головного потоку про зміну даних і відображення результатів у вікні. Оцінити переваги та особливості багатопотокової організації програми.

Індивідуальне завдання (Варіант №4)

Написати програму, яка запускає новий потік при натисканні лівої кнопки миші. Потік виводить зростаючу числову послідовність разом із поточними координатами курсора. При натисканні правої кнопки миші завершується той потік, координати якого найближчі до положення курсора.

Теоретичні відомості

Процес є середовищем виконання програми з власним адресним простором і ресурсами. Потік є послідовністю виконання всередині процесу. Потоки одного процесу використовують спільну пам’ять, але мають власний стек, регістри та стан виконання. Планувальник операційної системи розподіляє між ними процесорний час.

Функція CreateThread створює потік і передає параметр його стартовій функції. Дескриптор HANDLE дає змогу очікувати завершення потоку. Для узгодженого завершення можна встановити ознаку зупинки, дочекатися повернення з потокової функції через WaitForSingleObject і закрити дескриптор викликом CloseHandle.

Подія є об’єктом синхронізації, що повідомляє про настання певної умови. CreateEvent створює подію, а SetEvent переводить її у сигнальний стан. Подія з автоматичним скиданням після задоволення очікування одного потоку повертається в несигнальний стан.

Хід роботи

Програму реалізовано мовою C++ у проєкті Lab3. Файл Lab3.cpp містить створення вікна, цикл обробки повідомлень і реакції на мишу. Файли Worker.h та Worker.cpp визначають клас Worker, який зберігає дескриптор та ідентифікатор потоку, координати натискання, лічильник, час початку роботи й атомарну ознаку завершення.

Створення потоку

У повідомленні WM_LBUTTONDOWN з lParam отримуються координати натискання в клієнтській області. Функція CreateWorker створює об’єкт Worker і додає його до g_workers — вектора std::shared_ptr<Worker>. Метод Worker::CreateWorker викликає CreateThread, передаючи адресу StartThread та вказівник на створений об’єкт.

Функція StartThread приводить параметр LPVOID до Worker* і викликає ThreadCounter. Кожен об’єкт зберігає координати свого створення, тому результати різних потоків відображаються в різних місцях вікна.

Робота лічильника

Метод ThreadCounter виконує цикл, доки атомарна ознака m_exit має значення false. На кожній ітерації потік очікує 50 мс і за std::chrono::steady_clock обчислює кількість повних секунд від моменту створення об’єкта. Отримане число записується в m_counter. Отже, лічильник визначається за минулим часом, а не за кількістю ітерацій циклу.

Якщо значення лічильника змінилося, потік викликає SetEvent(g_redrawEvent). Головний потік очікує як цю подію, так і повідомлення Windows за допомогою MsgWaitForMultipleObjects. Після сигналу викликається InvalidateRect, а повідомлення вікна обробляються через PeekMessage, TranslateMessage і DispatchMessage.

Відображення та керування потоками

У WM_PAINT головний потік перебирає g_workers і формує текст для кожного об’єкта у форматі «лічильник | x: координата, y: координата». Функція DrawTextA виводить рядок у точці створення відповідного потоку. Координати фіксуються під час натискання миші. Малювання виконується в головному потоці, а робочі потоки оновлюють лічильники та сигналізують про зміни.

Вибір найближчого потоку

Повідомлення WM_RBUTTONDOWN передає координати правого натискання до EndClosestWorker. Для кожного об’єкта обчислюється евклідова відстань між його точкою створення та точкою натискання за допомогою std::hypot. Обирається об’єкт із найменшою відстанню. Якщо список порожній, завершення потоку не виконується.

Для вибраного об’єкта викликається метод Worker::ExitThread. Він установлює m_exit у true, очікує завершення через WaitForSingleObject і закриває дескриптор через CloseHandle. Після цього std::erase видаляє об’єкт із вектора. Метод класу ExitThread реалізує узгоджену зупинку; однойменна функція WinAPI у ньому не викликається.

Порядок роботи з програмою

1. Запустити додаток і натиснути ліву кнопку миші у кількох різних точках клієнтської області. Для кожної точки створюється окремий потік зі своїм лічильником.

2. Спостерігати за зростанням чисел. Потоки, створені в різний час, мають різні початкові моменти відліку, а їхні координати залишаються прив’язаними до точок створення.

3. Натиснути праву кнопку поблизу одного з написів. Програма завершує найближчий потік і прибирає його напис. Решта об’єктів залишаються у списку та продовжують роботу.



Рисунок 1 – Вікно програми

Висновок

У лабораторній роботі реалізовано динамічне створення потоків натисканням миші та завершення вибраного потоку за відстанню до курсора. Опрацьовано CreateThread, події WinAPI, очікування завершення і звільнення дескрипторів. Розподіл обчислення лічильників та малювання між робочими і головним потоками дає змогу поєднати паралельну роботу з обробкою подій інтерфейсу.